

REGISTRO DE CERTIFICADOS ACADÉMICOS EN REMIX, PASO A PASO

Su primer contrato: escrito, probado, desplegado en Sepolia y verificado en el explorador.
Al final, cualquier persona podrá comprobar un diploma sin cuenta y sin pagar.

MODALIDAD	DURACIÓN	RED	META
En parejas	80 min en clase	Remix VM y Sepolia	Contrato verificado + 18 pruebas

CÓMO LEER ESTA GUÍA

Está escrita para que puedan hacer el laboratorio **solos**, aunque sea la primera vez que abren Remix. Cada paso dice qué hacer, qué deberían ver y cómo saber que funcionó. Avancen en orden: cada paso supone que el anterior salió bien. Si algo no coincide con lo descrito, vayan a «**Errores frecuentes**» (sección 14). La interfaz de Remix cambia de vez en cuando: si un botón no está exactamente donde dice la guía, busquen el nombre de la acción; la lógica no cambia.

01 Qué van a construir, y por qué

[LEER PRIMERO](#)

Un registro donde la oficina de registro de la universidad publica el **hash** de cada diploma que emite. Una empresa que recibe el PDF de un diploma calcula su hash y le pregunta al contrato: ¿lo emitió la universidad?, ¿sigue vigente? La respuesta no depende de llamar a nadie ni de confiar en el PDF que le enviaron.

Objetivo: escribir, desplegar y verificar un primer contrato funcional en una red de prueba. **Resultado de aprendizaje:** RA3 (programar contratos inteligentes en Solidity).

FUNCIÓN	QUÉ HACE	QUIÉN LA LLAMA
<code>emitir(hash, programa)</code>	Registra el hash de un diploma nuevo con su programa y la fecha del bloque.	Solo el emisor
<code>revoke(hash, motivo)</code>	Marca un diploma como revocado. No lo borra: queda la historia.	Solo el emisor
<code>verificar(hash)</code>	Devuelve si es válido, el programa, la fecha y si fue revocado. Es <code>view</code> : gratis.	Cualquiera
<code>cambiarEmisor(nueva)</code>	Traspasa el rol de emisor a otra dirección.	Solo el emisor

En la cadena **no** va el nombre, ni la cédula, ni ningún dato personal. Solo el hash del archivo. La cadena es pública e inmutable, y la Ley Estatutaria 1581 de 2012 da a toda persona derecho a actualizar, rectificar y pedir la supresión de sus datos. Un hash del archivo completo no identifica a nadie por sí solo: quien tiene el PDF puede verificar; quien no lo tiene, no aprende nada. Tampoco usen un diploma real para el laboratorio: el repositorio trae uno ficticio.

SEGURIDAD

- Solo Sepolia y la Remix VM. La billetera del curso nunca se usa con dinero real.
- La frase de recuperación no se escribe en Remix, ni en un archivo, ni en un chat. Nadie legítimo la pide.
- Antes de confirmar cualquier firma: qué red, a qué dirección, cuánto. No se firma lo que no se entiende.
- El contrato es material de clase, sin auditar.

02 Qué se entrega

EVIDENCIA	DETALLE	PESO
Contrato verificado	Dirección de su contrato en Sepolia, con la marca de código verificado en <code>sepolia.etherscan.io</code> .	35 %
Tres transacciones	Hashes de: una emisión, una revocación y un intento fallido de emitir desde una cuenta que no es el emisor.	30 %
Prueba de alteración	Los dos hashes (original y alterado) y las capturas de <code>verificar</code> con cada uno, desde el explorador.	20 %
Una respuesta	¿Por qué el contrato revoca en lugar de borrar? Un párrafo, con referencia a su propia transacción de revocación.	15 %

Un documento por pareja, con la plantilla de la sección 16, antes de la Sesión 7. Sin dirección de contrato que abra en el explorador, el laboratorio no se califica.

03 Preparación desde cero

5 MINUTOS

NECESITAN	CÓMO COMPRUEBAN QUE LO TIENEN
Navegador con la billetera del curso	Chrome, Edge, Brave o Firefox con la billetera de la Sesión 2 instalada y desbloqueada.
Red Sepolia y saldo	La billetera muestra la red Sepolia y un saldo mayor que cero (el punto de control de la Sesión 5). Para desplegar y usar el contrato basta con unos 0,003 ETH de prueba.
Dos billeteras en la pareja	El intento fallido se hace desde la billetera del compañero. Si solo una tiene saldo, pasen un poco (0,001 ETH de prueba) de una a otra.
El andamiaje	El archivo <code>andamiaje/s06/CertificadosUSB.sol</code> del repositorio <code>laboratorios-evm</code> , abierto en un editor o en el navegador para copiarlo.
Opcional: Node.js 22	Para calcular el hash con los scripts y correr las pruebas. <code>node --version</code> debe decir <code>v22...</code> . Sin Node, usen los hashes de la sección 07.

Los archivos del laboratorio en el repositorio:

ARCHIVO	PARA QUÉ
<code>andamiaje/s06/CertificadosUSB.sol</code>	La versión de trabajo, con siete TODO. Compila tal como está.
<code>test/s06/CertificadosUSB.test.js</code>	18 pruebas que definen el comportamiento correcto.
<code>scripts/s06/diploma-de-prueba.pdf</code>	Un diploma ficticio, sin datos de ninguna persona.
<code>scripts/s06/hash-documento.js</code>	Calcula el keccak256 de los bytes de un archivo.
<code>scripts/s06/alterar-un-byte.js</code>	Crea una copia del archivo con un solo bit cambiado.

04 Paso 1 · Remix desde cero

10 MINUTOS

1

ABRIR REMIX

Un entorno completo dentro del navegador

Vayan a `https://app.remix.live` **en el mismo navegador donde está la billetera** (Remix se mudó a esa dirección en septiembre de 2026; `remix.ethereum.org` todavía abre). Remix no pide cuenta ni instalar nada. **Lo primero, antes de escribir nada:** en el panel del compilador elijan la versión `0.8.28+commit.7893614a`. Si dejan la que viene por omisión, que es más nueva, cualquier archivo del curso falla con `ParserError: Source file requires different compiler version`, porque los archivos fijan la versión a propósito. A la izquierda hay una barra de íconos: cada ícono es un complemento (*plugin*). Los tres que usan hoy son el **explorador de archivos**, el **compilador de Solidity** y **Deploy & Run Transactions**. Abajo, la **terminal** muestra cada transacción. **Cómo sé que funcionó:** ven un espacio de trabajo con las carpetas `contracts`, `scripts` y `tests`.

2

CREAR EL ARCHIVO

Pegar el andamiaje

En el explorador de archivos, clic derecho sobre `contracts` → *New File* → nómbrenlo `CertificadosUSB.sol`. Copien **todo** el contenido de `andamiaje/s06/CertificadosUSB.sol` y péguenlo. **Lean el archivo antes de seguir.** Tiene el estado (`struct Certificado`, `emisor`, `emitidos`, `el mapping`), los siete errores, los tres eventos y cinco bloques con comentarios `// TODO`.

3

CONFIGURAR EL COMPILADOR

Las mismas opciones que usarán al verificar

Ícono **Solidity Compiler**:

- *Compiler*: `0.8.28+commit.7893614a`. Elijanla exacta aunque Remix proponga una más nueva: es la versión congelada del curso.
- *Advanced Configurations*: activar **Enable optimization** con **200** ejecuciones. *EVM Version*: dejar la predeterminada (`default`).

Anoten estas tres opciones: el paso de verificación en el explorador exige repetirlas idénticas.

4

COMPILAR

Antes de escribir una sola línea

Clic en **Compile CertificadosUSB.sol** (o `Ctrl + S`).

Salida esperada: una marca verde sobre el ícono del compilador. Pueden aparecer **advertencias amarillas** («Unused function parameter»): son normales, porque los parámetros todavía no se usan. Desaparecen al completar los `TODO`.

Si aparece un error rojo: no se copió el archivo completo, o la versión elegida es menor que 0.8.28. El andamiaje compila tal como viene.

El ciclo, después de cada TODO: compilar → pestaña Deploy & Run con *Environment* en **Remix VM** → *Deploy* → probar a mano con la prueba indicada. Avanzar de uno en uno ahorra la mitad del tiempo de depuración. Cada vez que despliegan se crea un contrato **nuevo y vacío**: usen siempre el último de *Deployed Contracts*.

1

TODO 1 · MODIFICADOR soloEmisor

La puerta de todas las funciones que escriben

Qué hace: si quien llama (`msg.sender`) no es el `emisor` , revierte con `NoEsEmisor(msg.sender)` . Si sí es, sigue.

Qué escribir: una línea `if (...) revert NoEsEmisor(msg.sender);` **antes** del `_;` que ya está. El `_;` marca dónde se inserta el cuerpo de la función protegida: no lo borren.

Cómo probarlo: después del TODO 2 (sin constructor, el emisor es la dirección cero y nadie podría emitir).

2

TODO 2 · CONSTRUCTOR

Quién queda a cargo

Qué hace: se ejecuta una sola vez, al desplegar. Guarda a quien despliega como emisor y lo anuncia.

Qué escribir: `emisor = msg.sender;` y `emit EmisorCambiado(address(0), msg.sender);`

Cómo probarlo: desplegar y pulsar el botón azul `emisor` : debe mostrar la misma dirección que el selector *Account*. Luego cambien de *Account*, llamen `emitir` con cualquier hash: la terminal debe mostrar el error `NoEsEmisor` (esto prueba el TODO 1).

3

TODO 3 · VALIDACIONES DE emitir

Rechazar todo lo inválido antes de tocar el estado

Qué hace: tres comprobaciones, en este orden: hash en cero → `HashVacio()` ; programa vacío → `ProgramaVacio()` ; hash ya registrado → `YaEmitido(hashDocumento)` .

Pistas: el hash en cero es `bytes32(0)` . Un string no tiene `.length` : conviértanlo con

`bytes(programa).length` . Para saber si ya existe, lean el campo `existe` del certificado guardado en `_certificados[hashDocumento]` (toda clave de un mapping «existe» y devuelve ceros: por eso hace falta ese campo).

Cómo probarlo: emitir con hash

`0x0000000000000000000000000000000000000000000000000000000000000000` → debe fallar con `HashVacio` .

4

TODO 4 · EFECTOS DE emitir

Guardar, contar, avisar

Qué hace: guarda el certificado, suma uno al contador y emite el evento.

Pistas: la fecha es `uint64(block.timestamp)` ; guárdenla en una variable local para usar el mismo valor en el struct y en el evento. Un struct se crea así: `Certificado({ programa: programa, fechaEmision: ahora, revocado: false, existe: true })` . Luego `emitidos += 1`; y emit `CertificadoEmitido(hashDocumento, programa, ahora)`;

Cómo probarlo: emitir el hash de prueba

`0x87daf6ef2771bae4d21b7fb938774a35b9bd4a1a6989dd253c6694e9dcbcd455` con programa Ingeniería de Sistemas; emitidos debe dar 1 . Emitirlo otra vez: debe fallar con `YaEmitido` . En la terminal, despliegan la transacción exitosa: el campo `logs` muestra `CertificadoEmitido` .

5

TODO 5 · revocar

Marcar sin borrar

Qué hace: busca el certificado; si no existe → `NoExiste(hashDocumento)` ; si ya está revocado → `YaRevocado(hashDocumento)` ; si no, lo marca y emite `CertificadoRevocado(hashDocumento, motivo)` .

La trampa: `Certificado` storage `c = _certificados[hashDocumento]`; — con storage, `c` apunta al dato guardado y `c.revocado = true`; queda escrito. Con `memory` modificarían una copia que desaparece al terminar: compila, no da error, y no revoca nada.

Cómo probarlo: revocar el hash emitido con motivo `prueba` ; revocarlo otra vez → `YaRevocado` ; revocar un hash nunca emitido → `NoExiste` .

6

TODO 6 · verificar

La consulta que hace el mundo

Qué hace: devuelve cuatro valores: `valido` (existe y no revocado), `programa` , `fechaEmision` y `revocado` .

Pista: lean el certificado con `storage` (aquí no se modifica, solo evita copiarlo) y devuelvan la tupla `return (c.existe && !c.revocado, c.programa, c.fechaEmision, c.revocado)`;

Cómo probarlo: el botón `verificar` es azul (view): responde al instante, sin transacción. Antes de revocar: `true` , el programa y una fecha. Después: `false` y `revocado = true` . Con un hash nunca emitido: `false` y `fecha 0` .

7

TODO 7 · cambiarEmisor

Traspasar el rol

Qué hace: rechaza `address(0)` con `DireccionCero()` , emite `EmisorCambiado(emisor, nuevo)` antes de cambiarlo (para que el evento lleve el emisor anterior) y actualiza `emisor` .

Cómo probarlo: copien otra dirección de la lista *Account* (ícono de copiar), llamen `cambiarEmisor` con ella; luego intenten emitir con la cuenta original → `NoEsEmisor` ; cambien a la cuenta nueva y emitan → funciona.

06 Paso 3 · La batería de pruebas a mano en la Remix VM

ANTES DE IR A
SEPOLIA

Con los siete TODO completos, desplieguen una vez más en la **Remix VM** (gratis e instantánea) y recorran esta tabla completa. Si todas las filas dan lo esperado, el contrato está listo para una red pública. Equivocarse aquí no cuesta nada; en Sepolia, cada error es una transacción pagada y un contrato que ya no se puede corregir.

#	ACCIÓN (CUENTA)	RESULTADO ESPERADO
1	<code>emisor</code>	La cuenta con la que desplegaron.
2	<code>emitir(0x87da...d455, "Ingenieria de Sistemas")</code> (emisor)	Transacción exitosa (✓ verde en la terminal). <code>emitidos</code> = 1.
3	El mismo <code>emitir</code> otra vez	Falla con <code>YaEmitido</code> .
4	<code>emitir(0x00...00, "x")</code>	Falla con <code>HashVacio</code> .
5	<code>emitir(otro hash, "")</code>	Falla con <code>ProgramaVacio</code> .
6	<code>emitir(otro hash, "x")</code> desde OTRA cuenta	Falla con <code>NoEsEmisor</code> y la dirección de esa cuenta.
7	<code>verificar(0x87da...d455)</code>	<code>true</code> , <code>Ingenieria de Sistemas</code> , <code>fecha ≠ 0</code> , <code>false</code> .
8	<code>verificar(0xc89d...3951)</code> (el alterado)	<code>false</code> , texto vacío, <code>0</code> , <code>false</code> .
9	<code>revocar(0x87da...d455, "prueba")</code> , y otra vez	La primera funciona; la segunda falla con <code>YaRevocado</code> .
10	<code>verificar(0x87da...d455)</code>	<code>false</code> , el programa, la misma fecha, <code>true</code> .
11	<code>emitir(0x87da...d455, "x")</code>	Falla con <code>YaEmitido</code> : un hash revocado no se reutiliza.
12	<code>cambiarEmisor(0x0000...0000)</code>	Falla con <code>DireccionCero</code> .

07 Paso 4 · El hash del diploma

5 MINUTOS

Lo que se registra no es el diploma sino su huella: el keccak256 de **todos los bytes** del archivo. En la carpeta `laboratorios-evm`:

```
node scripts/s06/hash-documento.js scripts/s06/diploma-de-prueba.pdf

archivo : scripts/s06/diploma-de-prueba.pdf
tamaño  : 2.226 bytes
keccak   : 0x87daf6ef2771bae4d21b7fb938774a35b9bd4a1a6989dd253c6694e9dcbed455
```

Ahora alteren un solo bit (el script crea una copia; el original no se toca) y calculen de nuevo:

```
node scripts/s06/alterar-un-byte.js scripts/s06/diploma-de-prueba.pdf
node scripts/s06/hash-documento.js scripts/s06/diploma-de-prueba-alterado.pdf

keccak : 0xc89d02a9a7f8b6660f3567945455456ed1e2b96cdd0637371b863b2c0f913951
```

El PDF alterado se abre igual y se ve idéntico, pero su hash no tiene nada que ver: es el efecto avalancha de la Sesión 2. Esos dos valores son los que se usan en Sepolia.

VARIANTES

Con un documento propio: cualquier PDF que ustedes creen, **sin datos personales**, sirve igual; su hash será distinto al de esta guía. **Sin Node.js:** usen el diploma del repositorio y los dos valores de arriba, que son exactos. **Nunca** un diploma real.

1

ENTORNO

De la máquina virtual a una red pública

Deploy & Run → *Environment*: elijan la opción de la billetera del navegador. Según la versión de Remix se llama **Browser Extension** (y dentro, MetaMask) o **Injected Provider – MetaMask**. La billetera abre una ventana pidiendo **conectar** la cuenta a Remix: acepten solo la cuenta del curso.

Cómo sé que funcionó: debajo de *Environment* aparece la red: **Sepolia (11155111)**. En *Account* aparece su dirección con su saldo real de ETH de prueba.

Si aparece otra red: cámbienla a Sepolia en la billetera; Remix se actualiza solo.

2

DESPLEGAR

La primera transacción que crea un contrato

Confirman que en *Contract* está seleccionado `CertificadosUSB` y pulsen **Deploy**. La billetera muestra la transacción. **Antes de confirmar, revisen:**

- **Red:** Sepolia. Si dice cualquier otra, rechacen.
- **Tipo:** despliegue de contrato («Contract deployment»), sin destinatario.
- **Valor:** 0 ETH.
- **Costo estimado:** del orden de 0,0005 a 0,002 ETH de prueba. El despliegue consume unas 555 000 unidades de gas (medido en la red local); el precio depende de la tarifa base del momento.

Confirman. **Salida esperada:** en 12 a 30 segundos la terminal de Remix muestra la transacción con ✓ y en *Deployed Contracts* aparece `CERTIFICADOSUSB AT 0x...`. **Copien esa dirección** (ícono de copiar) y guárdenla: es la evidencia principal.

3

USAR EL CONTRATO EN SEPOLIA

Las transacciones de la evidencia

Ahora cada botón naranja abre la billetera y cuesta gas real de prueba. En orden:

1. `emitir` con el hash original y el programa. Confirman. Guarden el hash de la transacción (clic en la línea de la terminal → *transaction hash*).
2. `verificar` con el hash original (debe dar `true`) y con el alterado (`false`). Son lecturas: no abren la billetera.
3. **Intento fallido:** en Remix, el compañero abre Remix en **su** navegador con **su** billetera, pega y compila el mismo código, pega la dirección del contrato en *At Address* y llama `emitir` con otro hash. Remix avisará *Gas estimation failed* y ofrecerá enviarla igual: **envíenla a propósito**. Quedará en la cadena con Status Fail y costará muy poco. Guarden su hash.
4. Con la cuenta del emisor, `revoke` el diploma con un motivo que **no** contenga datos personales (por ejemplo, `prueba de laboratorio`). Guarden el hash.

Abran `https://sepolia.etherscan.io` y peguen la dirección del contrato. La página muestra la transacción de creación (*Contract Creation*) y las de uso. Abran la de `emitir` → pestaña **Logs**:

PARTE	QUÉ DEBEN VER
Address	La dirección de su contrato.
Topics[0]	<code>0x774ca613bc69539f740c09dc7a428006e870e9cb3dff99a89fbd49088e2c3fd6</code> : el <code>keccak256</code> de <code>CertificadoEmitido(bytes32,string,uint64)</code> . Es igual para todas las parejas, porque es la firma del evento.
Topics[1]	El hash del diploma, completo: es el parámetro <code>indexed</code> .
Data	El programa y la fecha, codificados. Antes de verificar el código se ven en hexadecimal; después, el explorador los traduce.

Abran también la transacción fallida: *Status* dice `Fail` y, cuando el código esté verificado, el motivo aparece como `NoEsEmisor` . No tiene logs: una ejecución revertida no deja eventos.

10 Paso 7 · Verificar el código fuente

10 MINUTOS

Verificar es publicar el código fuente y dejar que el explorador compruebe que, compilado con las mismas opciones, produce **exactamente** el bytecode desplegado. Sin eso, nadie puede saber qué hace su contrato.

1

OPCIÓN A · FORMULARIO DEL EXPLORADOR

Verify and Publish en Etherscan

En la página del contrato → pestaña **Contract** → **Verify and Publish**. Primera pantalla:

- *Compiler Type*: **Solidity (Single file)**.
- *Compiler Version*: **v0.8.28+commit.7893614a**.
- *Open Source License Type*: **MIT License (MIT)** (la del encabezado SPDX).

Segunda pantalla: peguen el código **exacto** que compilaron en Remix, completo. *Optimization*: **Yes**; en la configuración avanzada, *Runs*: **200** y *EVM Version*: **default**. *Constructor Arguments*: vacío (el constructor no recibe parámetros). Completen la verificación anti-bots y envíen.

Salida esperada: «Successfully generated Bytecode and ABI for Contract Address». En la pestaña *Contract* aparece una marca verde y las subpestañas *Read Contract* y *Write Contract*.

2

OPCIÓN B · DESDE REMIX

El complemento Contract Verification

En Remix, *Plugin Manager* → activar **Contract Verification**. Permite verificar en Sourcify sin clave de API (Etherscan, en cambio, exige una clave propia). Elijan la red Sepolia, peguen la dirección del contrato y verifiquen. Sourcify es un verificador independiente y de código abierto; si el explorador no muestra la marca verde, usen la opción A para la evidencia.

3

LA PRUEBA FINAL

Verificar un diploma sin billetera

En el explorador → *Contract* → **Read Contract** → *verificar*: peguen el hash original y pulsen *Query*; luego el alterado. Tomen captura de ambos resultados. **Esto es lo que se construyó**: cualquier persona, desde cualquier computador, sin cuenta y sin pagar, comprueba el diploma.

11 Las pruebas automáticas del andamiaje

PLAN B Y COMPROBACIÓN FINAL

El repositorio trae 18 pruebas en `test/s06/CertificadosUSB.test.js`. Sirven de **plan B** si la sala bloquea Remix, y de **comprobación final** para todos: si su código pasa las 18, el comportamiento es el correcto. Requieren Node.js 22 y `npm install` hecho una vez (Sesión 5).

1

COPIAR SU VERSIÓN SOBRE LA DEL REPOSITORIO

Qué archivo prueban las pruebas

Las pruebas compilan lo que haya en `contracts/s06/CertificadosUSB.sol`, que en el repositorio es la solución de referencia. Para probar **su** trabajo, reemplácenlo en su copia local: o con el andamiaje (para trabajar ahí) o pegando lo que escribieron en Remix.

```
cp andamiaje/s06/CertificadosUSB.sol contracts/s06/CertificadosUSB.sol      (macOS,  
Linux, Git Bash)  
copy andamiaje\s06\CertificadosUSB.sol contracts\s06\CertificadosUSB.sol    (Windows cmd  
/ PowerShell)
```

2

CORRER

Leer el rojo como una lista de tareas

```
npx hardhat test test/s06/CertificadosUSB.test.js
```

Con el andamiaje sin tocar: 4 passing y 14 failing . Pasan las tres de *verificar* que esperan ceros y la de privacidad; el resto falla porque los TODO están vacíos. Cada prueba en rojo dice qué comportamiento falta, por ejemplo quien despliega queda como emisor → TODO 2.

Con los siete TODO bien: 18 passing . Ese es el criterio de terminado.

PARA RECUPERAR LA SOLUCIÓN DE REFERENCIA

Si trabajan sobre un clon del repositorio: `git checkout contracts/s06/CertificadosUSB.sol` . No suban la solución a su repositorio de entrega. El despliegue con script a Sepolia (con las claves en el almacén cifrado: `npx hardhat keystore set ...` , nunca en archivos) se hace en la Sesión 8.

12 Puntos de control

¿VOY BIEN?

MINUTO	DEBERÍAN TENER
10	El andamiaje compilado en Remix con 0.8.28 y optimización 200.
25	TODO 1, 2, 3 y 4: emitir funciona y <code>emitidos</code> = 1 en la Remix VM.
45	Los siete TODO y la tabla de la sección 06 recorrida sin sorpresas.
50	Los dos hashes del diploma calculados (o tomados de la sección 07).
65	Contrato desplegado en Sepolia y las cuatro transacciones hechas: emisión, lecturas, intento fallido, revocación.
80	Código verificado, Logs revisados y capturas de Read Contract.

13 Cómo saber que terminaron

- La dirección del contrato abre en `sepolia.etherscan.io` con la marca verde de código verificado.
- La pestaña *Logs* muestra `CertificadoEmitido` y `CertificadoRevocado` emitidos por **su** contrato.
- Hay una transacción con *Status* `Fail` desde una cuenta que no es el emisor.
- *Read Contract* → `verificar` responde `false` para el hash alterado, y `false` con `revocado = true` para el original.
- Opcional pero recomendado: `18 passing` con su código en Hardhat.

SÍNTOMA	CAUSA Y SOLUCIÓN
Error rojo al compilar el andamiaje	Archivo incompleto o versión de compilador menor a 0.8.28.
«Gas estimation failed» al emitir	Una validación revierte: hash en cero, programa vacío, hash ya emitido o cuenta distinta del emisor. Lean el error que muestra la terminal.
Nadie puede emitir, ni siquiera quien desplegó	Falta el TODO 2: el emisor quedó en la dirección cero.
revocar no cambia nada	Usaron <code>Certificado memory c</code> en lugar de <code>storage</code> : modificaron una copia.
verificar dice válido un hash nunca emitido	Olvidaron exigir <code>c.existe</code> .
Los certificados «desaparecieron»	Redesplegaron: cada Deploy crea un contrato nuevo y vacío. Usen el último o <i>At Address</i> con la dirección anterior.
«Operator == not compatible with types string»	No se comparan strings con <code>==</code> . Usen <code>bytes (programa) .length == 0</code> para el vacío.
Remix no ve la billetera	Recarguen Remix, desbloqueen la billetera, acepten la conexión. Debe ser el mismo navegador.
La billetera dice «saldo insuficiente»	Pidan fondos de prueba al docente. No intenten desplegar en otra red.
La transacción queda «pendiente» mucho tiempo	Tarifa muy baja o red congestionada. Esperen o usen «acelerar» en la billetera; no reenvíen la misma acción dos veces.
La verificación en el explorador falla	Versión del compilador, optimización, runs o EVM Version distintos al despliegue, o el código pegado no es exactamente el desplegado (un espacio cambia los metadatos).
En Hardhat: «HH... not found» o «No Hardhat config»	La terminal no está en la carpeta <code>laboratorios-evm</code> , o falta <code>npm install</code> .

15 Preguntas de reflexión

CON SU CONTRATO DELANTE

Respondan mirando **su** contrato desplegado y **sus** transacciones. La primera es obligatoria; elijan dos más.

1. **Obligatoria:** ¿por qué el contrato revoca en lugar de borrar? Usen como argumento lo que muestra el explorador sobre su transacción de revocación y lo que devuelve `verificar` después.
2. En su transacción de emisión, ¿cuánto gas se usó? Compárenlo con el de la revocación. Con lo visto en la Sesión 5, ¿qué explica la diferencia?
3. ¿Cuánto gas consumió su intento fallido y cuánto pagó el compañero? ¿Qué habría pasado si hubieran firmado un límite de gas menor que ese consumo?
4. Busquen en la pestaña *Logs* de su emisión el valor de `Topics[1]`. ¿Por qué ese parámetro está marcado `indexed` y el programa no?
5. Supongan que alguien roba la clave del emisor. Con su contrato desplegado delante, ¿qué puede hacer y qué no? ¿Qué quedaría registrado?
6. Su contrato guarda `programa` como texto. ¿Hay algún escenario en el que eso sea un dato personal? ¿Y el motivo de revocación?

16 Plantilla de entrega

```
EVIDENCIA · LABORATORIO 06 · CERTIFICADOS EN REMIX
Pareja: Nombre 1 · Nombre 2                      Fecha: DD/MM/AAAA

1. CONTRATO
  Dirección: https://sepolia.etherscan.io/address/0x...   (verificado: sí)
  Compilador: 0.8.28 · optimización 200 · EVM default

2. TRANSACCIONES
  Emisión    : https://sepolia.etherscan.io/tx/0x...   gas usado: ...
  Revocación: https://sepolia.etherscan.io/tx/0x...   gas usado: ...
  Fallida    : https://sepolia.etherscan.io/tx/0x...   motivo: NoEsEmisor

3. PRUEBA DE ALTERACIÓN
  Hash original : 0x...   verificar → true / false (tras revocar)
  Hash alterado : 0x...   verificar → false
  [capturas de Read Contract con ambos]

4. PRUEBAS AUTOMÁTICAS (opcional): resultado de npx hardhat test

5. RESPUESTAS: la obligatoria y dos más de la sección 15

6. DECLARACIÓN DE USO DE IA: para qué se usó, o «no se usó»
```

17 Lista de chequeo antes de entregar

✓	COMPROBACIÓN
☐	La dirección del contrato abre en el explorador de Sepolia y tiene la marca de verificado.
☐	Los tres hashes de transacción abren y son de SU contrato.
☐	La transacción fallida viene de una cuenta distinta del emisor.
☐	Hay capturas de <code>verificar</code> con el hash original y con el alterado.
☐	Ningún dato personal en el contrato: ni en <code>programa</code> , ni en el motivo de revocación, ni en el documento hasheado.
☐	Ninguna frase de recuperación, clave privada ni URL con clave en el documento o en un repositorio.
☐	Respuesta obligatoria y dos más; declaración de uso de IA.

18 Rúbrica

CRITERIO	PESO	EXCELENTE (5,0)	ACEPTABLE (3,5)	INSUFICIENTE (<3,0)
Contrato verificado	35 %	Desplegado y verificado; se comporta como la tabla de la sección 06.	Desplegado sin verificar, o con un comportamiento incorrecto.	Sin dirección verificable.
Transacciones	30 %	Las tres, de su contrato, con el motivo del fallo identificado.	Falta una o el motivo no se identifica.	Hashes que no abren o de otro contrato.
Alteración	20 %	Dos hashes y dos capturas coherentes desde el explorador.	Capturas solo desde Remix.	Ausente.
Respuesta	15 %	Argumenta con su propia transacción de revocación y con la confianza que da la historia.	Correcta pero genérica.	Ausente o incorrecta.

19 Referencias

- Remix IDE: <https://app.remix.live> · documentación: <https://remix-ide.readthedocs.io>
- Remix · Contract Verification: https://remix-ide.readthedocs.io/en/latest/contract_verification.html
- Solidity, documentación oficial: <https://docs.soliditylang.org>
- Sourcify: <https://sourcify.dev>
- Explorador de Sepolia: <https://sepolia.etherscan.io>
- Ley Estatutaria 1581 de 2012: <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981>